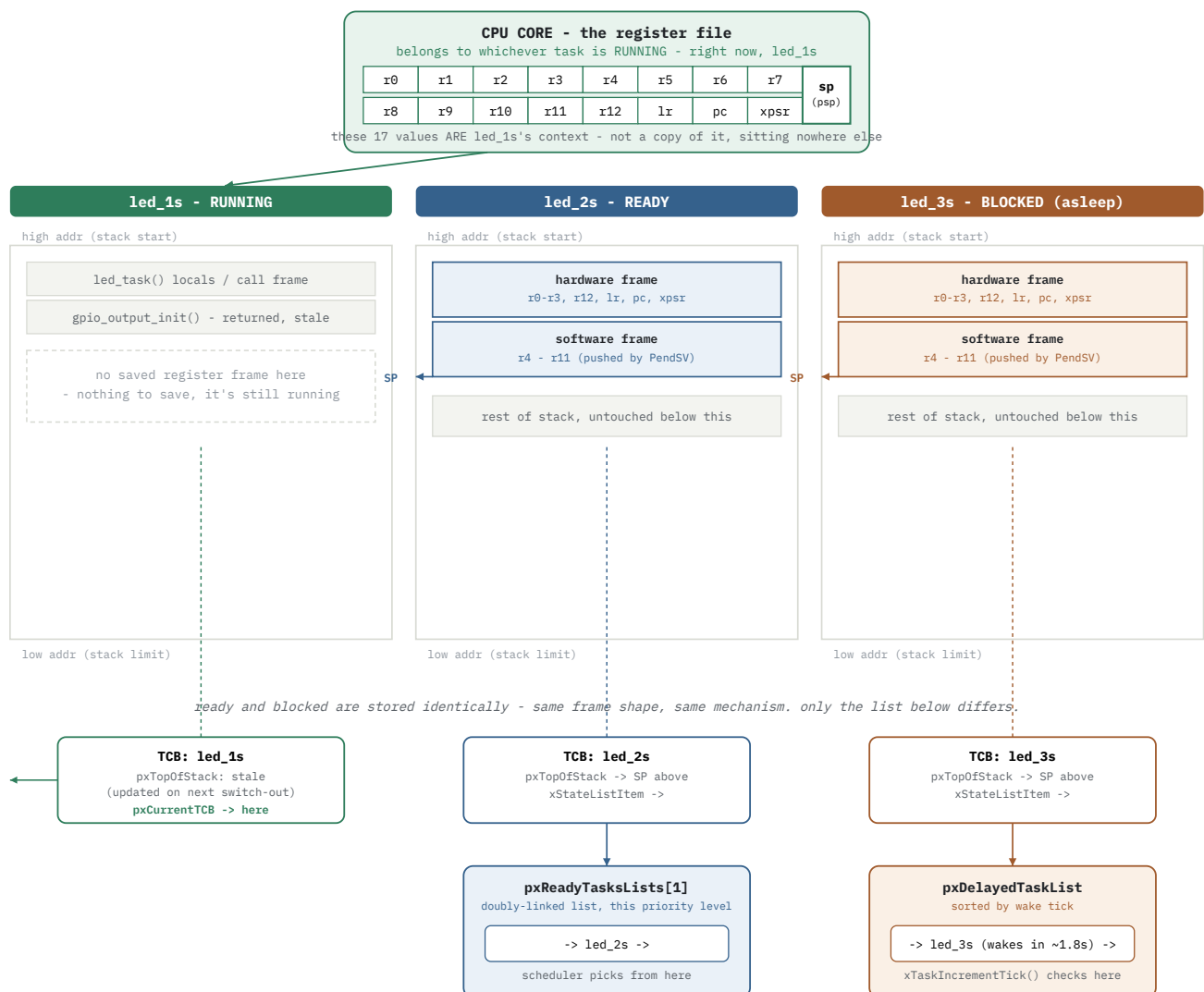


stm32f103 workspace notes

Where a task's context actually lives

A snapshot of your own three-thread-blink at one instant: led_1s is running, led_2s is ready, led_3s is asleep in vTaskDelay(). Same three tasks, three different answers to "where is this task's context right now."

running - context is live in the CPU ready - context saved, waiting for the CPU
blocked - context saved, waiting for a timeout



A running task's context is the CPU's registers, full stop - nothing is copied anywhere while it runs. A non-running task's context is a frozen snapshot sitting on its own stack, found via TCB->pxTopOfStack. Ready vs. blocked is not a difference in *how* the context is stored - it's only a difference in which list the TCB is threaded into.

What's actually inside a saved frame

THE EIGHT-PLUS-EIGHT WORDS BEHIND LED_2S'S SP

Zooming into one of those "software frame / hardware frame" blocks from above - this is the literal word-by-word memory layout, in the exact order `PendSV_Handler` pushes and pops it.

higher address

xPSR	pushed automatically by hardware, on the way into ANY exception
PC	return address
LR	
R12	
R3	
R2	
R1	
R0	
R11	pushed by hand, in <code>PendSV_Handler</code> 's own <code>"stmdb r0!, {r4-r11}"</code>
R10	
R9	
R8	
R7	
R6	
R5	
R4	SP → = <code>TCB->pxTopOfStack</code>

16 words, roughly 64 bytes - that's an entire task's execution state, in full, sitting quietly in RAM until the scheduler decides it's this task's turn again.

Same mechanism, different list

The one thing worth saying twice: `led_2s` and `led_3s` have *identical* saved frames above - same shape, same fields, same mechanism. Nothing about either stack tells you which one is ready and which is sleeping. Look back at the two boxes at the bottom of

the first diagram - that distinction lives entirely in which linked list the TCB is threaded into, nowhere else.

The wake-up path. Every SysTick, `xTaskIncrementTick()` checks the head of `pxDelayedTaskList` - it's kept sorted by wake time, so it only ever has to look at the front. When `led_3s`'s tick arrives, it's unlinked from the delayed list and relinked into `pxReadyTasksLists[]` - nothing about its saved stack frame changes in that move. It becomes eligible to run again purely by changing which list it's in.

stm32f103 workspace notes · continues from the SysTick / PendSV / SVC notes